Lab 1: First Web API using .NET Core

Aim

To understand the concepts of RESTful Web Services, Web API, Microservices, HTTP Request and Response, HTTP Action Verbs, HTTP Status Codes, configuration files, and create a simple ASP.NET Core Web API using Controller and Swagger.

1. RESTful Web Service

Definition

A **RESTful Web Service** is a web service that follows the **REST (Representational State Transfer)** architecture. It allows communication between client and server using the HTTP protocol.

REST treats everything as a **resource**, identified by a URI (Uniform Resource Identifier).

Features of REST Architecture

1. Representational State Transfer

Resources are represented in formats such as:

- JSON
- XML

Example JSON:

```
{
   "id":1,
   "name":"Laptop",
   "price":75000
}
```

2. Stateless

Each request is independent.

The server does not store client information between requests.

Example

GET /api/Values

The next request

POST /api/Values

does not remember the previous GET request.

3. Client-Server Architecture

Client and server are independent.

4. Uniform Interface

Uses standard HTTP methods.

- GET
- POST
- PUT
- DELETE

5. Cacheable

Responses can be cached for faster performance.

2. Web API

Definition

A **Web API (Application Programming Interface)** is a service that allows different software applications to communicate over HTTP.

It exchanges data in JSON or XML format.

Advantages

- Fast
- Lightweight
- Platform Independent
- Supports JSON
- Easy Integration

3. Microservice

Definition

A **Microservice** is a small independent service responsible for one business functionality.

Example

E-Commerce System

```
|
├── Product Service
├── Customer Service
├── Payment Service
├── Order Service
└── Delivery Service
```

Each service has

- Separate Database
- Separate API
- Separate Deployment

Difference between Web Service and Web API

Web Service	Web API
Supports SOAP and REST	Mostly REST
XML response	JSON/XML response
Slower	Faster
More complex	Lightweight
Platform Independent	Platform Independent

---

4. HTTP Request

Definition

The request sent by the client to the server.

Contains

- URL
- HTTP Method
- Headers
- Body

Example

GET /api/Values

5. HTTP Response

Definition

The response returned by the server.

Contains

- Status Code
- Response Body
- Headers

Example

```
[
  "Value1",
  "Value2",
  "Value3"
]
```

6. HTTP Action Verbs

GET

Used to retrieve data.

Example

```
[HttpGet]
public IActionResult Get()
{
    return Ok(new string[]
    {
        "Value1",
        "Value2",
        "Value3"
    });
}
```

POST

Used to insert data.

```
[HttpPost]
public IActionResult Post()
{
    return Ok("POST Executed");
}
```

PUT

Used to update data.

```
[HttpPut]
public IActionResult Put()
{
    return Ok("PUT Executed");
}
```

DELETE

Used to delete data.

```
[HttpDelete]
public IActionResult Delete()
{
    return Ok("DELETE Executed");
}
```

HTTP Action Verbs Summary

Verb	Purpose
GET	Retrieve Data
POST	Insert Data
PUT	Update Data
DELETE	Delete Data

7. HTTP Status Codes

200 OK

Request executed successfully.

Example

return Ok();

201 Created

Resource created successfully.

Example

return Created();

400 Bad Request

Client sends invalid request.

Example

return BadRequest();

401 Unauthorized

User authentication failed.

Example

return Unauthorized();

404 Not Found

Requested resource not available.

Example

return NotFound();

500 Internal Server Error

Unexpected server error.

Example

return StatusCode(500);

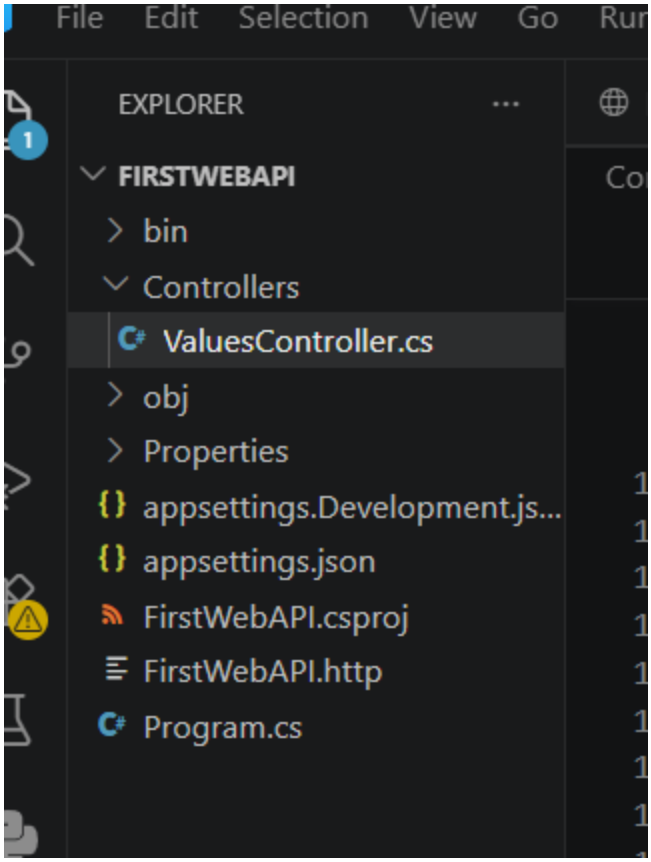
HTTP Status Codes Summary

Status Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
500	Internal Server Error

---

8. Structure of ASP.NET Core Web API

FirstWebAPI



9. ApiController

The `[ApiController]` attribute enables API-specific features such as:

- Automatic model validation
- Parameter binding
- Better error responses

Example

```
[ApiController]
public class ValuesController : ControllerBase
{
}
```

---

10. ControllerBase

`ControllerBase` is the base class for Web API controllers.

It provides methods like:

- Ok()
- BadRequest()
- NotFound()
- Unauthorized()

Example

```
public class ValuesController : ControllerBase
{
}
```

11. IActionResult

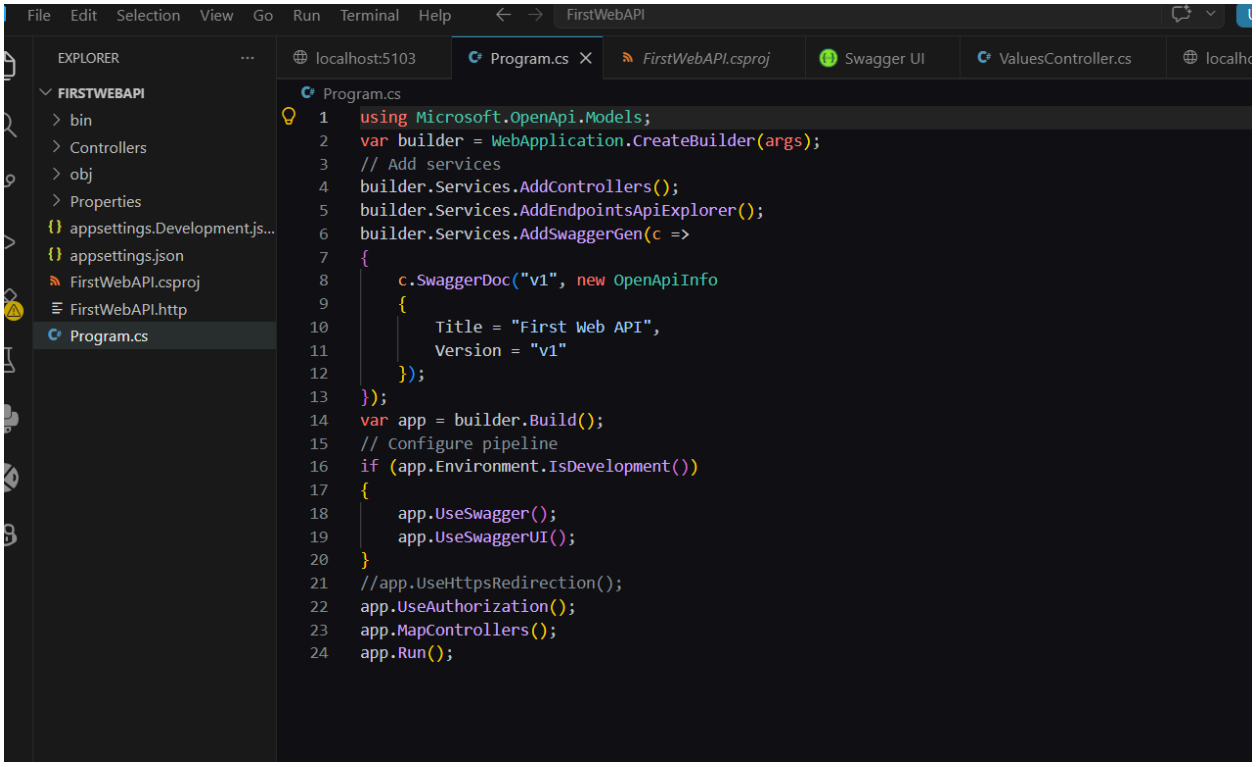
Represents different HTTP responses returned by an action method.

Examples

```
return Ok();
return BadRequest();
return NotFound();
return Unauthorized();
```

12. Configuration Files

Program.cs



RouteConfig.cs (.NET Framework)

Used to configure Web API routing.

Example

config.MapHttpAttributeRoutes();

WebApiConfig.cs (.NET Framework)

Registers Web API routes.

Example

```
config.Routes.MapHttpRoute(
    name: "DefaultApi",
    routeTemplate: "api/{controller}/{id}"
);
```

---

13. Swagger

Swagger is used to test Web APIs without writing client code.

Features

- Displays all APIs
- Sends requests
- Shows responses
- Generates API documentation